An Internship Report on

FACE RECOGNITION USING PYTHON AND OPENCV

Submitted To

School of Computer Science and Engineering
IILM University, Greater Noida, U.P.

In partial fulfilment of the requirement for the award of the degree of
Bachelor of Technology in Computer Science and Engineering (B.Tech CSE)

By

Name of Student: Devanshi Jain
Roll Number: 2410030209
Batch: 2026-2027

2026-27

CANDIDATE'S DECLARATION

I, Devanshi jain, do hereby declare that the internship report titled "FACE RECOGNITION USING PYTHON AND OPENCV" has been completed by me to fulfil the requirement for the award of the degree of Bachelor of Technology in Computer Science and Engineering. All the references have been quoted and I have not taken as such any material from any other source. I affirm that this report is my own work and has not been submitted to any other institute for any degree/diploma requirement, and I shall be solely responsible for any kind of copyright violation in this regard.

Signature: devanshi
Student Name: devanshi jain
Roll No.:2410030209

ACKNOWLEDGEMENT

I am using this opportunity to express my gratitude to everyone who supported me throughout this internship. I am thankful for their aspiring guidance, invaluable constructive criticism, and friendly advice throughout the internship period. I am sincerely grateful to them for sharing their truthful and illuminating views on a number of issues related to this project.

I would like to express my sincere gratitude to Dr. Anurag Shrivastava (Program Manager, Codec Technologies) for their guidance and support during the course of this internship at Codec Technologies Pvt. Ltd.. I would also like to thank HOD and the faculty of the School of Computer Science and Engineering, IILM University, for their continuous encouragement.

I express my gratitude to my parents for their blessings.

Signature:
Student Name: Devanshi jain
Roll No.:2410030209

INTERNSHIP COMPLETION CERTIFICATE

TABLE OF CONTENTS

CHAPTER 4

PROJECT DESCRIPTION

1. Introduction

1. Project Overview

The project titled **"Face Recognition Using Python and OpenCV"** is a computer-vision-based application developed to detect and recognize human faces from images. The project demonstrates how image-processing techniques and machine-learning-based face recognition can be combined to create an automated identification system.

The project is implemented using the Python programming language and the OpenCV library. OpenCV is used for image reading, image conversion, face detection, image processing, and face recognition. The project uses a **Haar Cascade classifier** to detect faces and an **LBPH (Local Binary Pattern Histogram) face recognizer** to recognize faces from the prepared training dataset.

The project follows a sequence of operations. First, facial images are organized into folders inside the `Training_Data` directory. The `create_csv.py` module processes these images, detects faces using the Haar Cascade classifier, extracts the detected facial regions, and creates the training information. The resulting training image paths and labels are stored in a CSV file.

The main program, `Face_Recog.py`, loads the generated training information, reads the training images, converts them into grayscale, and trains an LBPH face-recognition model. After training, a test image is supplied to the program. The system detects faces in the test image and uses the trained LBPH model to predict the corresponding label.

2. Need for the Project

Manual identification of people from images can be time-consuming, particularly when a large number of images need to be checked. Computer vision provides a way to automate this process.

A face-recognition system can be useful in applications such as:

- Identity verification

- Attendance systems
- Access-control applications
- Image organization
- Security-related applications
- Smart surveillance prototypes
- Human-computer interaction
- Educational demonstrations of artificial intelligence and computer vision

This project focuses on demonstrating the fundamental concepts rather than developing a production-grade biometric system.

4.1.3 Basic Working Principle

The complete process can be represented as:

Training Images -> Face Detection -> Face Extraction -> Label Generation -> LBPH Training -> Test Image -> Face Detection -> Face Prediction -> Recognition Result

The repository README confirms this overall pipeline and identifies Haar Cascade detection, training-data preparation, CSV label generation, LBPH training, and recognition testing as the main features.

2. Organization Profile

1. About Codec Technologies Pvt. Ltd.

This internship was undertaken at **Codec Technologies Pvt. Ltd.**, an organization engaged in providing industry-oriented technical training and internship programs to engineering students. Codec Technologies conducts skill-development programs in collaboration with the **National Internship Portal**, and its internship programs are approved under **AICTE and ICAC**. Codec Technologies is also recognized as a **Google for Education Partner**.

The internship was completed under the role of **Python Developer Intern**, as part of a **1-month AICTE & ICAC approved internship program**, conducted from **25/07/2026 to 25/08/2026**, under the supervision of **Dr. Anurag Shrivastava (Program Manager)**.

2. Internship Role and Domain

As a Python Developer Intern, the internship focused on practical, hands-on development using Python, with an emphasis on real-world application building. The project assigned during the internship — **Face Recognition Using Python and OpenCV** — falls within the domain of computer vision and image processing, and was used to build practical skills in Python-based software development.

3. Project Repository Profile

The source code for the project developed during the internship is maintained and hosted on GitHub under the repository **`vivekraj-02/image-recognition`**, which contains the complete implementation of the face-recognition system.

The repository contains the source code and supporting files required for the project, including:

- `Face_Recog.py`
- `create_csv.py`
- `Training_Data/`
- `haarcascade_face.xml`
- `requirements.txt`
- `test.jpg`
- `README.md`
- `LICENSE`

The repository is available publicly and uses the MIT License.

4.2.4 Development Environment

The development environment consists primarily of:

- Python programming language
- OpenCV computer-vision library
- NumPy
- Pandas
- Haar Cascade classifier
- LBPH face recognizer

- CSV-based training-data management
- GitHub for source-code hosting

The project can be executed from a Python environment after installing the dependencies specified in `requirements.txt`. The README recommends creating a virtual environment and installing the project dependencies before execution.

3. Problem Statement

1. Existing Problem

Identifying individuals from photographs manually requires a person to visually inspect images and compare facial characteristics. This process becomes inefficient when the number of images increases.

A computer-based face-recognition system is therefore required to automatically:

1. Read an input image.
2. Detect faces present in the image.
3. Extract the detected facial regions.
4. Compare the facial features against a previously prepared training dataset.
5. Predict the identity label associated with the detected face.
6. Display the recognition result.

4.3.2 Proposed Problem Solution

The proposed project solves the problem by implementing a two-stage computer-vision pipeline:

STAGE 1 - FACE DETECTION

A Haar Cascade classifier is used to identify the location of faces within an image.

STAGE 2 - FACE RECOGNITION

The detected facial region is provided to an LBPH face recognizer trained using previously prepared facial images.

The system therefore separates "where is the face?" from "which trained label does this face most closely correspond to?"

4.3.3 Technical Problem

The project must handle several practical issues:

- Different image sizes
- Color images versus grayscale images
- Multiple faces in a single image
- Missing or invalid image files
- Empty training datasets
- Incorrect training-data organization
- Variations in facial appearance
- Recognition confidence/distance

The implementation includes error handling for missing training data, invalid training images, unreadable test images, and test images in which no face is detected.

4. Project Objectives

1. Primary Objective

The primary objective is to develop a Python-based face-recognition prototype capable of detecting faces in images and predicting their corresponding labels using a trained LBPH model.

2. Specific Objectives

The project objectives are:

7. To understand the fundamentals of computer vision.
8. To implement face detection using OpenCV.
9. To use a Haar Cascade classifier for detecting faces.
10. To organize facial images into a structured training dataset.
11. To automatically extract facial regions from training images.
12. To generate labels for different individuals.

13. To store training information in CSV format.
14. To preprocess images into grayscale format.
15. To train an LBPH face-recognition model.
16. To process a test image.
17. To detect one or more faces in the test image.
18. To predict the label associated with each detected face.
19. To display the recognition result visually.
20. To implement basic exception and error handling.
21. To understand the complete workflow of an image-recognition application.

The repository's main recognition program creates the training data, reads `train_faces.csv`, loads valid images, converts them to grayscale, creates an LBPH recognizer, trains the recognizer, and then performs prediction on the test image.

5. Scope of the Project

1. Current Scope

The current project focuses on image-based face recognition using a predefined training dataset.

The current system supports:

- Training images organized by person
- Haar Cascade face detection
- Cropping of detected faces
- Training-data generation
- CSV-based label management
- Grayscale image preprocessing
- LBPH model training
- Test-image face detection
- Face prediction
- Bounding-box visualization
- Prediction-label display

- Confidence/distance display

The repository README specifically states that each person should have an individual folder inside `Training_Data` and that the model is trained using extracted face crops stored in `Training_Faces`.

4.5.2 Functional Scope

The system performs the following functions:

Input:

- Training images
- Test image

Processing:

- Face detection
- Face extraction
- Data labeling
- Image preprocessing
- Model training
- Face prediction

Output:

- Detected face
- Bounding rectangle
- Numerical predicted label
- Confidence/distance value

4.5.3 Limitations

The current implementation also has some limitations:

- It primarily works with still images rather than continuous video.
- Recognition depends heavily on the quality and variety of the training dataset.
- The numerical label does not directly represent a person's name unless a separate label-to-name mapping is maintained.

- Haar Cascade may be less robust than modern deep-learning-based face detectors under difficult conditions.
- LBPH is an introductory/traditional recognition method and may not perform as well as modern deep-learning approaches on challenging datasets.
- The project does not currently provide a complete graphical user interface.
- The project does not currently include a database for storing user profiles.

These limitations provide opportunities for future development.

4.5.4 Future Scope

Future versions can include:

- Webcam-based real-time recognition
- Real-time video processing
- Name-based identity mapping
- Graphical user interface
- Database integration
- Attendance management
- Improved dataset management
- Multiple-camera support
- Advanced deep-learning face recognition
- Better handling of lighting and pose variations
- Performance and accuracy evaluation
- Deployment as a web or desktop application

6. Technologies and Tools Used

1. Python

Python is the primary programming language used in the project. It provides libraries and tools suitable for image processing, data manipulation, machine learning, and rapid application development.

2. OpenCV

OpenCV is the main computer-vision library used by the project.

The implementation imports OpenCV using:

```
import cv2
```

It is used for:

- Reading images
- Converting images to grayscale
- Detecting faces
- Drawing rectangles
- Displaying images
- Performing LBPH recognition

The main program uses OpenCV's Haar Cascade classifier and LBPH face recognizer.

4.6.3 Haar Cascade Classifier

The project contains the file:

```
haarcascade_face.xml
```

This classifier is loaded using OpenCV's `CascadeClassifier`.

Its purpose is to detect the location of faces in images.

The detection process uses parameters including:

- `scaleFactor = 1.1`
- `minNeighbors = 5`
- `minSize = (30, 30)`

These parameters determine how the classifier searches for facial regions.

4.6.4 LBPH Face Recognition

The project uses:

```
cv2.face.LBPHFaceRecognizer_create()
```

LBPH stands for Local Binary Pattern Histogram.

The LBPH method represents local texture patterns in facial images and uses these patterns for recognition. In this project, grayscale facial images are used to train the recognizer, and the trained model predicts the label of a detected test face.

4.6.5 NumPy

NumPy is used for numerical operations and for converting the list of labels into an integer array required by the recognizer.

The project imports NumPy using:

```
import numpy as np
```

The labels are converted using an integer NumPy array before model training.

4.6.6 Pandas

Pandas is used to read the generated CSV training information.

The project uses:

```
pd.read_csv('train_faces.csv')
```

The CSV information is then converted into values that can be processed by the training function.

4.6.7 CSV

CSV is used as an intermediate format for storing:

- Training image path
- Numerical label

This makes it easier for the training program to associate each facial image with the correct identity label.

4.6.8 Argparse

The `argparse` module allows the user to supply the test image as a command-line argument.

The repository's usage is:

```
python Face_Recog.py <path_to_test_image>
```

4.6.9 GitHub

GitHub is used to host and share the project source code. The repository contains the project's source files, documentation, training-data structure, classifier, requirements, and license.

7. System Architecture

1. Overall Architecture

The system consists of two major phases:

PHASE 1 - TRAINING

```
Training_Data
      |
      v
Read Images
      |
      v
Haar Cascade Face Detection
      |
      v
Face Extraction
      |
      v
Training_Faces
      |
      v
Label Generation
      |
      v
train_faces.csv
      |
      v
Grayscale Conversion
      |
      v
LBPH Model Training
```

PHASE 2 - RECOGNITION

```
Test Image
     |
     v
Image Reading
     |
     v
Grayscale Conversion
     |
     v
Haar Cascade Face Detection
     |
     v
Face Crop
     |
```

```
     v
Trained LBPH Model
     |
     v
Prediction
     |
     v
Label + Confidence
     |
     v
Display Result
```

The two phases are implemented primarily through `create_csv.py` and `Face_Recog.py`.

4.7.2 Detailed Architecture

MODULE 1 - TRAINING DATA

The `Training_Data` directory contains source images grouped according to individuals.

Example:

```
Training_Data/
|
+-- Person_1/
|   +-- image1.jpg
|   +-- image2.jpg
|   +-- image3.jpg
|
+-- Person_2/
    +-- image1.jpg
    +-- image2.jpg
    +-- image3.jpg
```

The repository documentation specifies that every person should have their own folder.

MODULE 2 - FACE DETECTION

The Haar Cascade classifier scans each image and identifies facial regions.

MODULE 3 - FACE EXTRACTION

The detected facial region is cropped and saved as a training face.

MODULE 4 - LABEL GENERATION

A numerical label is associated with each person's training images.

MODULE 5 - CSV GENERATION

The paths and labels are written into:

```
train_faces.csv
```

MODULE 6 - MODEL TRAINING

The grayscale training images are passed to the LBPH face recognizer.

MODULE 7 - TEST IMAGE

The user supplies an image to be tested.

MODULE 8 - RECOGNITION

The system detects the face and passes the facial crop to the trained recognizer.

MODULE 9 - RESULT

The predicted label and confidence/distance value are displayed on the image.

8. Methodology

1. Step 1 - Requirement Analysis

The first step is to identify the requirements of the face-recognition system.

Functional Requirements:

- The system should accept facial images.
- The system should detect faces.
- The system should create training data.
- The system should train a recognition model.
- The system should accept a test image.
- The system should predict the label of detected faces.

- The system should display the recognition result.

Non-Functional Requirements:

- Easy to execute
- Simple project structure
- Reasonable processing speed
- Maintainable Python code
- Error handling for invalid inputs

4.8.2 Step 2 - Dataset Organization

Training images are placed inside `Training_Data`.

Each individual is represented by a separate directory.

This directory-based organization makes it possible to associate images with labels during dataset preparation.

4.8.3 Step 3 - Face Detection

The Haar Cascade classifier is loaded:

```
cv2.CascadeClassifier('haarcascade_face.xml')
```

The classifier is then used to detect faces.

The test process converts the image to grayscale before detection. The detected facial regions are returned as rectangular coordinates `(x, y, w, h)`.

4.8.4 Step 4 - Face Extraction

After detecting a face, the corresponding region is extracted from the original image.

The extracted facial images form the actual training samples.

The repository identifies `Training_Faces/` as the location for cropped face images generated during training.

4.8.5 Step 5 - Label Generation

Each training image is associated with a numerical label.

For example:

```
Person A -> Label 0
Person B -> Label 1
Person C -> Label 2
```

The numerical labels are later supplied to the LBPH recognizer during training.

4.8.6 Step 6 - CSV Creation

The image paths and labels are stored in `train_faces.csv`.

Conceptually:

Image Path	Label
Training_Faces/image1.jpg	0
Training_Faces/image2.jpg	0
Training_Faces/image3.jpg	1
Training_Faces/image4.jpg	1

The main recognition script reads this CSV before training.

4.8.7 Step 7 - Image Preprocessing

The training images are loaded using OpenCV.

Each valid image is converted from BGR/color representation into grayscale.

This is important because LBPH works with intensity patterns rather than requiring full-color information.

The code performs grayscale conversion using:

```
cv2.cvtColor(image, cv2.COLOR_BGR2GRAY)
```

4.8.8 Step 8 - LBPH Model Creation

The face-recognition model is created using:

```
cv2.face.LBPHFaceRecognizer_create()
```

The grayscale images and their labels are then supplied to the model.

The code trains the recognizer using:

```
face_recognizer.train(images, np.array(labels, dtype=np.int32))
```

4.8.9 Step 9 - Test Image Input

The test image is provided through the command line.

Example:

```
python Face_Recog.py test.jpg
```

The program uses `argparse` to obtain the path of the test image.

4.8.10 Step 10 - Test Image Preprocessing

The test image is loaded and converted to grayscale.

If the image cannot be read, the program raises an error indicating that the test image could not be read.

4.8.11 Step 11 - Face Detection in Test Image

The Haar Cascade classifier searches the test image for faces.

The project uses:

- Scale factor: `1.1`
- Minimum neighbors: `5`
- Minimum face size: `30 x 30`

If no face is found, the program prints:

```
No face detected in the test image.
```

4.8.12 Step 12 - Prediction

For every detected face, the system extracts the face crop and passes it to:

```
face_recognizer.predict(face_crop)
```

The model returns:

- Predicted label
- Confidence/distance value

The implementation then displays this information on the image.

4.8.13 Step 13 - Result Visualization

A rectangle is drawn around the detected face.

The prediction is displayed above the rectangle in the following general format:

```
Label X (confidence value)
```

The processed image is then displayed in an OpenCV window titled "Face Recognition".

4.8.14 Step 14 - Error Handling

The project includes basic error handling. Examples include:

Empty Training Dataset: If no training data is available, an error is generated indicating that images need to be added to `Training_Data`.

Invalid Training Images: If no valid images can be loaded from the CSV, the program reports that no valid images were found.

Invalid Test Image: If the test image cannot be read, an appropriate error is raised.

No Face Detected: If no face is found in the test image, the system reports that no face was detected.

The main program catches exceptions and prints the corresponding error message before exiting.

9. Expected Outcomes

1. Technical Outcomes

After successful execution, the project is expected to:

- Read the training dataset.
- Detect faces from training images.
- Generate cropped face images.
- Generate training labels.
- Create `train_faces.csv`.
- Train an LBPH recognition model.
- Read the test image.
- Detect faces in the test image.
- Predict the corresponding numerical label.

- Display the detected face using a bounding box.
- Display the prediction and confidence/distance value.

4.9.2 Learning Outcomes

The project provides practical knowledge of:

- Python programming
- Computer vision
- Image preprocessing
- Face detection
- Haar Cascade classifiers
- LBPH face recognition
- NumPy
- Pandas
- CSV-based data handling
- Command-line arguments
- Exception handling
- GitHub-based project management

4.9.3 Practical Outcome

The final outcome is a working prototype that demonstrates the complete basic workflow of face recognition:

```
Input Dataset -> Face Detection -> Face Extraction -> Label
Generation ->
Model Training -> Test Image -> Face Detection -> Face Recognition ->
Prediction -> Displayed Result
```

4.9.4 Expected Benefits

The project demonstrates how computer vision can reduce manual effort in image-based identification tasks. It also provides a foundation for developing more advanced applications such as attendance systems, identity verification prototypes, image classification systems, and real-time face-recognition applications.

However, because this is a basic educational implementation, the system should not be treated as a high-security biometric identification solution without further testing, validation, privacy safeguards, and stronger recognition methods.

4.10 Certificates of Completion and Other Communication Mail Proof

CODEC TECHNOLOGIES

NATIONAL INTERNSHIP PORTAL

INTERNSHIP OFFER LETTER

Dear Devanshi Jain,

We are pleased to offer you a 1 Month Internship as Project Intern at Codec Technologies, a global platform dedicated to empowering learners and connecting diverse talent to create meaningful career opportunities worldwide. Codec Technologies delivers dedicated IT and business consultancy services across 27+ countries, empowering global innovation and strategic growth.

Internship Details:

- Designation : Python Developer Intern
- Location & Mode: Pan India, Hybrid
- Duration : 09/08/2026 to 09/09/2026
- Reporting to : Assigned Project Head(s)

This program provides an immersive experience, enabling you to gain industry-level knowledge and acquire valuable skills. Exceptional interns may also qualify for a scholarship award based on weekly performance reviews conducted by project supervisor.

If you accept this offer and the terms above, please complete your assigned projects and training tasks on our platform and via email.
Best regards,

Sincerely,
Dr. Anurag Shrivastava,
Codec Technologies India
NCS ID: E19E86-0116588288923
Date Of Issue - 09/08/2026

Dr. Anurag Shrivastava
Talent Acquisition Manager

Google for Education Partner

ISO 9001 CERTIFIED

www.codectechnologies.in support@codectechnologies.in Chandivali IT hub, Mumbai


CODEC
TECHNOLOGIES
NATIONAL
INTERNSHIP
PORTAL
CERTIFICATE
OF INTERNSHIP
This certificate is awarded to
Devanshi Jain
From Codec Technologies Pvt. Ltd.
In recognition of his/her efforts and achievements in completing the 1 Month AICTE & ICAC
Approved internship program as
Python Developer Intern
Conducted from 09/08/2026 to 09/09/2026
Google for Education
Partner
AICTE ID - CORPORATE6759d549ce59e1733940553
Dr. Anurag Shrivastava
Program Manager

CHAPTER 5

BIBLIOGRAPHY / REFERENCES

1. Devanshi jain. "image-recognition"
2. OpenCV Documentation. https://docs.opencv.org/
3. OpenCV Haar Cascade Classifier Documentation — Object Detection with Haar Cascades.
4. OpenCV LBPH Face Recognizer Documentation — cv2.face.LBPHFaceRecognizer_create().
5. NumPy Documentation. https://numpy.org/doc/
6. Pandas Documentation. https://pandas.pydata.org/docs/
7. Python argparse Module Documentation. https://docs.python.org/3/library/argparse.html
8. Codec Technologies Pvt. Ltd. — Internship communication and project allocation records.